

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Use Case

WhatNext Vision Motors is an automotive company focused on delivering innovative mobility solutions while improving customer satisfaction and operational efficiency. To achieve these objectives, the company has implemented a Salesforce-based system that simplifies vehicle management, improves the ordering experience, and automates essential business operations.

The primary objective of this project is to enhance the vehicle ordering process by introducing an automated dealer assignment mechanism. Based on the customer's address, the system identifies and suggests the nearest suitable dealer location. This functionality helps customers connect with an appropriate dealer more conveniently while reducing manual effort and improving the overall ordering experience.

Another important aspect of the system is vehicle stock validation. The application checks the availability of a vehicle before allowing an order to proceed. When the requested vehicle is unavailable, the system prevents the order from being processed as a confirmed purchase. This helps avoid incorrect bookings, minimizes customer inconvenience, and maintains accurate inventory information.

The project also includes an automated process for handling bulk vehicle orders. Through scheduled batch processing, the system examines order records and updates their status according to the available stock. Orders associated with vehicles that are available are updated to **Confirmed**, whereas orders without sufficient stock remain **Pending**. This automation ensures consistent order status management and provides better visibility into the order fulfillment process.

The Salesforce implementation is designed to improve the reliability and efficiency of vehicle sales operations. By combining customer management, dealer assignment, stock validation, and automated order processing, the system reduces manual errors and supports a smoother customer experience.

In addition, automation reduces repetitive administrative work for employees, allowing them to concentrate on customer support and other important business activities. The solution also provides a structured approach to managing vehicle inventory and orders, helping WhatNext Vision Motors respond more effectively to customer requirements and operational demands.

Requirements:

1. Salesforce CRM Implementation:

- Maintain vehicle information, inventory details, and dealer records within the Salesforce platform.
- Manage customer orders, test drive bookings, and vehicle service requests systematically.
- Implement automated processes to associate customer orders with the appropriate dealer based on location.

2. Process Automation:

- Validate vehicle stock availability before processing customer orders.
- Automatically identify and assign the suitable dealer using the customer's location.
- Configure automated email reminders for upcoming test drive appointments.

3. Apex and Triggers:

- Develop Apex triggers to implement business rules, including vehicle stock validation and order processing.
- Apply the trigger handler pattern to organize business logic into reusable, modular, and maintainable components.

4. Batch Jobs:

- Create Batch Apex functionality to process vehicle inventory records and handle stock updates.
- Configure scheduled processing for bulk order management and vehicle stock maintenance.

What You Will Learn:

1. Data Modeling
2. Fields and Relationships
3. Lightning App Builder
4. Record-Triggered Flows
5. Apex Classes and Triggers
6. Batch Apex
7. Scheduled Apex

Milestone 1:

Activity 1: Creating a Salesforce Developer Account

The first activity involves registering a Salesforce Developer account to create and configure the development environment required for the project.

1. Visit the Salesforce Developer registration page:
<https://developer.salesforce.com/signup>
2. Enter the required information in the sign-up form, including the following details:

Build enterprise-quality apps fast to bring your ideas to life

- Build apps fast with drag and drop tools
- Customize your data model with clicks
- Go further with Apex code
- Integrate with anything using powerful APIs
- Stay protected with enterprise-grade security
- Customize UI with clicks or any leading-edge web framework

Sign up for your Salesforce Developer Edition
A full-featured copy of the Platform, for free

Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial.

First Name*
Your first name

Last Name*
Your last name

Email*
Your email address

Role*
Your job role

Company*
Company Name

- 1) First name & Last name
- 2) Email
- 3) Role : Developer
- 4) Company : College Name
- 5) County : India
- 6) Postal Code : pin code
- 7) Username : should be a combination of your name and company

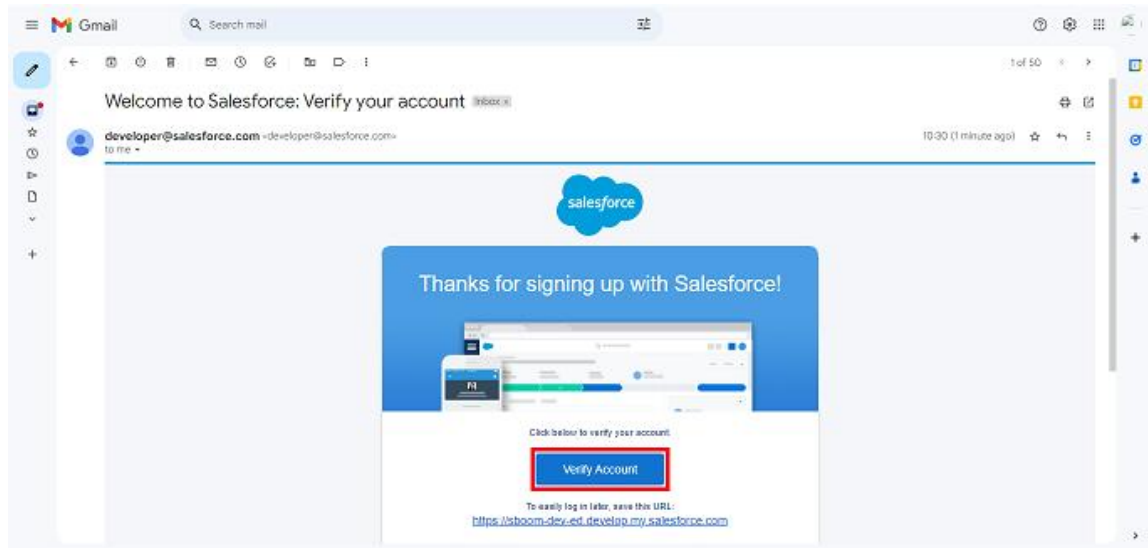
This need not be an actual email id, you can give anything in the format :

username@organization.com

Click on sign me up after filling these.

Activity 2: Account Activation

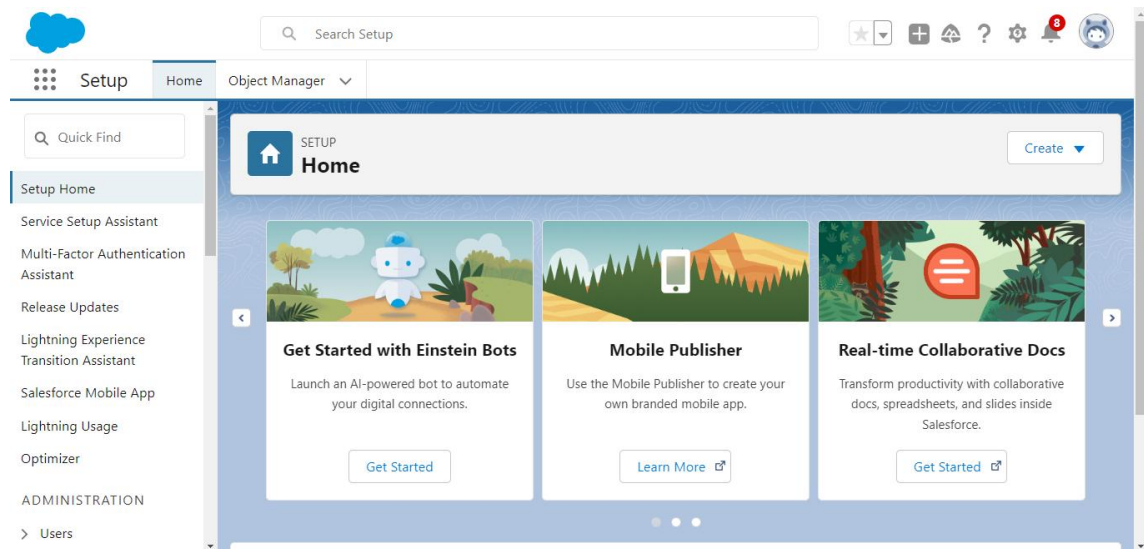
1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.

A screenshot of the 'Change Your Password' page in Salesforce. The page has a light blue background with a white form. The form contains the following elements: a heading 'Change Your Password', a prompt 'Enter a new password for lead@sb.com. Make sure to include at least:', three green checkmarks indicating password requirements (8 characters, 1 letter, 1 number), a 'New Password' field with a 'Good' status, a 'Confirm New Password' field with a 'Match' status, a 'Security Question' dropdown menu showing 'In what city were you born?', an 'Answer' field containing 'asdfghjkl', and a blue 'Change Password' button at the bottom. A red rectangular box highlights the entire form area.

4. Then you will redirect to your salesforce setup page.



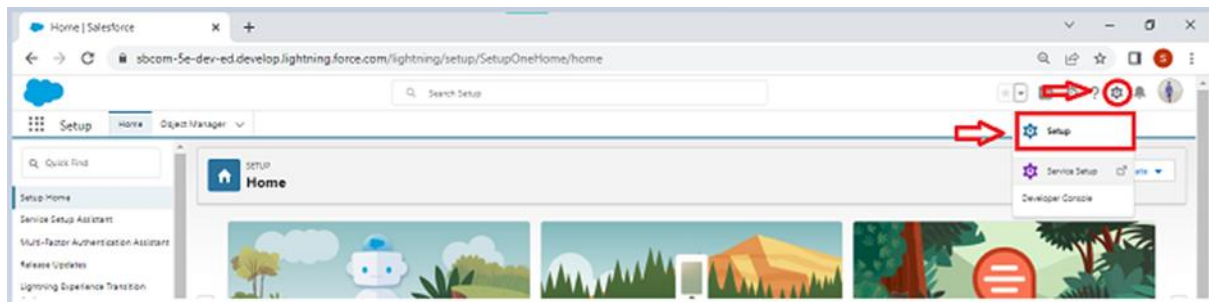
MileStone 2 :

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:

Click on gear icon → click setup.



Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



- 1) Enter the label name → Vehicle
- 2) Plural label name → Vehicles
- 3) Enter Record Name Label and Format
 - Record Name → Vehicle Name
 - Data Type → Text

A screenshot of the 'New Custom Object' form in Salesforce. The form is titled 'New Custom Object' and has a 'Custom Object Definition Edit' header. It contains several sections: 'Custom Object Information' with fields for 'Label' (Example: Account), 'Plural Label' (Example: Accounts), and 'Object Name' (Example: Account); 'Enter Record Name Label and Format' with fields for 'Record Name' (Example: Account Name) and 'Data Type' (Text). Red boxes highlight the 'Label', 'Plural Label', 'Object Name', 'Record Name', and 'Data Type' fields. A warning message is visible at the bottom: 'Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.'

2. Click on Allow reports,

3. Allow search → **Save**.

The screenshot shows the Salesforce Object Settings page for a custom object. The page is divided into several sections:

- Optional Features:** Contains checkboxes for 'Allow Reports', 'Allow Activities', 'Track Field History', 'Allow in Chatter Groups', and 'Enable Licensing'. A red arrow points to the 'Allow Reports' checkbox.
- Object Classification:** Explains that when settings are enabled, the object is classified as an Enterprise Application object, and when disabled, it is a Light Application object. It includes checkboxes for 'Allow Sharing', 'Allow Bulk API Access', and 'Allow Streaming API Access'.
- Deployment Status:** Includes radio buttons for 'In Development' and 'Deployed'.
- Search Status:** Explains that when this setting is enabled, users can find records of this object type when they search. It includes a checkbox for 'Allow Search'. A red arrow points to the 'Allow Search' checkbox.
- Object Creation Options (Available only when custom object is first created):** Includes checkboxes for 'Add Notes and Attachments related list to default page layout' and 'Launch New Custom Tab Wizard after saving this custom object'.

At the bottom of the page, there are three buttons: 'Save', 'Save & New', and 'Cancel'. The 'Save' button is highlighted with a red box, and a red arrow points to it. The 'Save & New' button is also highlighted with a red box and an arrow.

Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
 - 1) Enter the label name→ Vehicle Dealer
 - 2) Plural label name→ Vehicle Dealers
 - 3) Enter Record Name Label and Format
 - Record Name → Dealer Name
 - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

Activity 3: Creating Customer objects

The purpose of creating a Vehicle Customer object is to have detailed information about Vehicle Customer.

To create an object:

4. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
 - 4) Enter the label name→ Vehicle Customer
 - 5) Plural label name→ Vehicle Customers
 - 6) Enter Record Name Label and Format
 - Record Name → Vehicle Customer Name
 - Data Type → Text
5. Click on Allow reports,
6. Allow search → **Save**.

Activity 4: Creating Vehicle Order objects

The purpose of creating a Vehicle Order object is to have detailed information about Vehicle Order.

To create an object:

7. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
- 7) Enter the label name→ Vehicle Order
- 8) Plural label name→ Vehicle Orders
- 9) Enter Record Name Label and Format
 - Record Name → Vehicle Order Name
 - Data Type → Text
8. Click on Allow reports,

Allow search → **Save.**

Activity 5: Creating Vehicle Test Drive objects

The purpose of creating a Vehicle Test Drive object is to have detailed information about Vehicle Test Drive.

To create an object:

9. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
- 10) Enter the label name→ Vehicle Test Drive
- 11) Plural label name→ Vehicle Test Drives
- 12) Enter Record Name Label and Format
 - Record Name → Vehicle Test Drive Name
 - Data Type → Text
10. Click on Allow reports,
11. Allow search → **Save.**

Activity 6: Creating Vehicle Service Request objects

The purpose of creating a Vehicle Service Request object is to have detailed information about Vehicle Service Request.

To create an object:

12. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
- 13) Enter the label name→ Vehicle Service Request
- 14) Plural label name→ Vehicle Service Requests
- 15) Enter Record Name Label and Format
 - Record Name → Vehicle Service Request Name

- Data Type → Text
13. Click on Allow reports,

Allow search → **Save**.

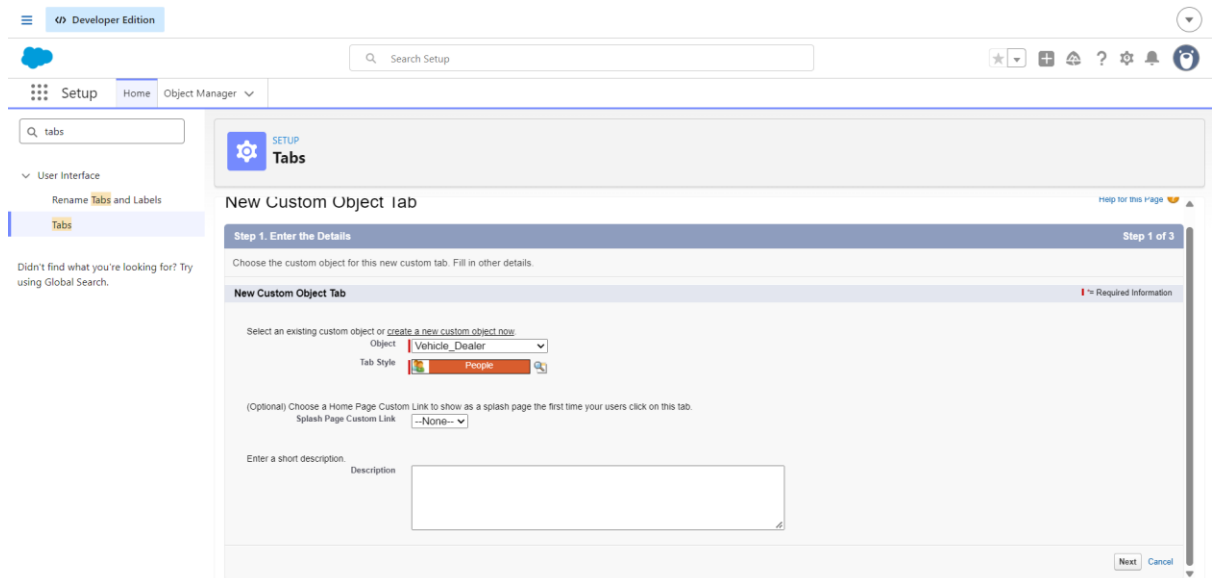
Milestone 3 - Tabs

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)
2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.

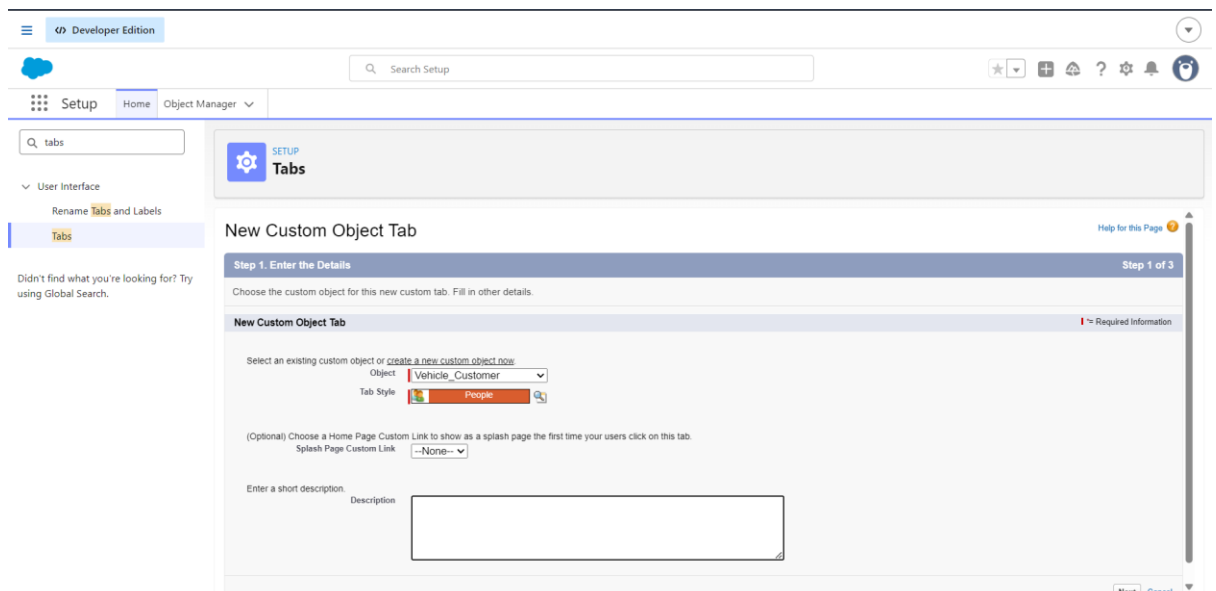
Activity 2: Creating a Custom Tab(Vehicle_Dealer)

3. Select Object(Vehicle_Dealer) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



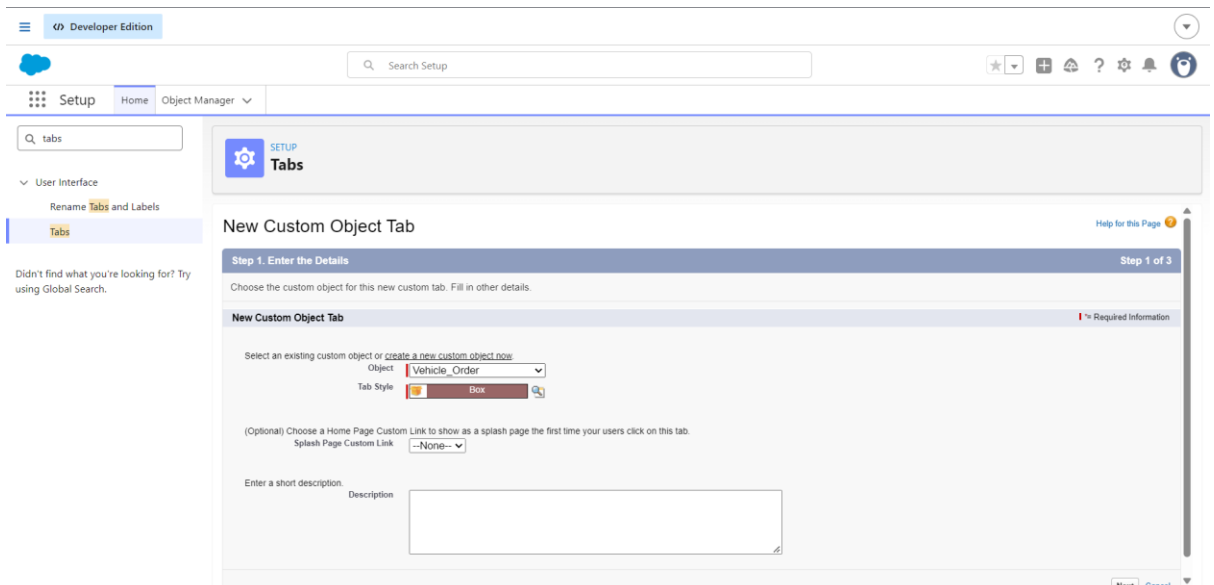
Activity 3: Creating a Custom Tab(Vehicle_Customer)

4. Select Object(Vehicle_Customer) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



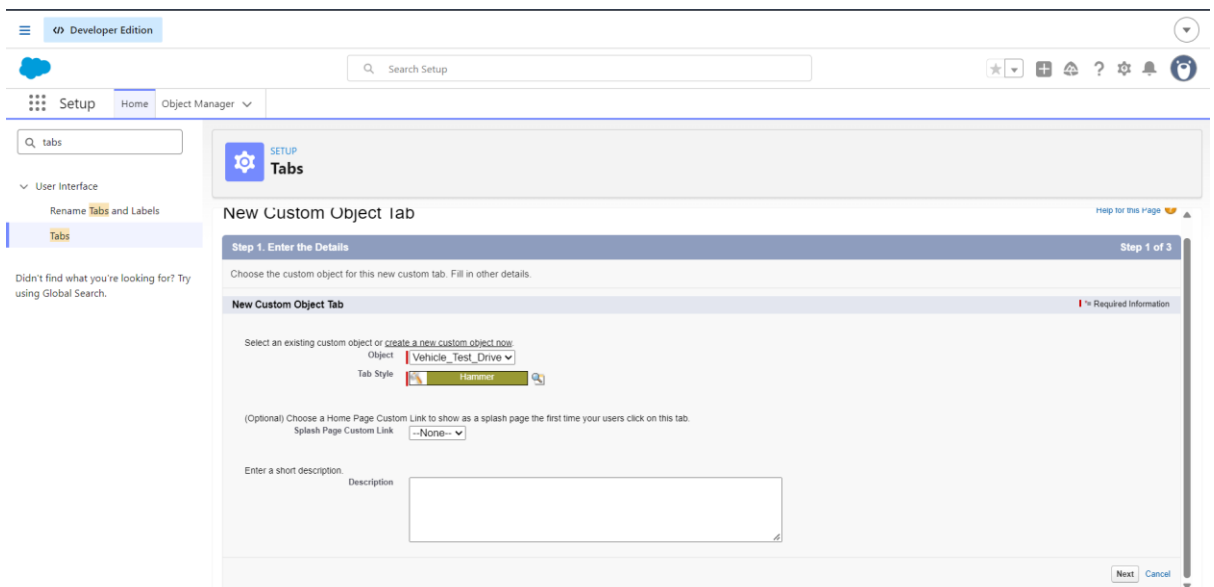
Activity 4: Creating a Custom Tab(Vehicle_Order)

5. Select Object(Vehicle_Order) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



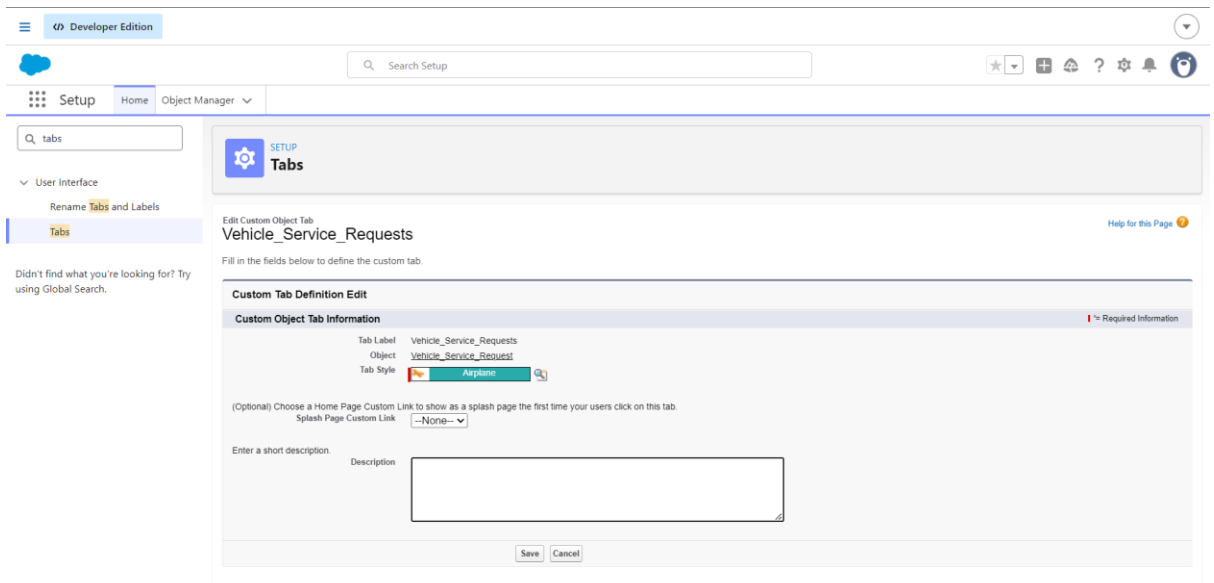
Activity 5: Creating a Custom Tab(Vehicle_Test_Drive)

6. Select Object(Vehicle_Test_Drive) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.



Activity 6: Creating a Custom Tab(Vehicle_Service_Request)

7. Select Object(Vehicle_Service_Request) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save&New.

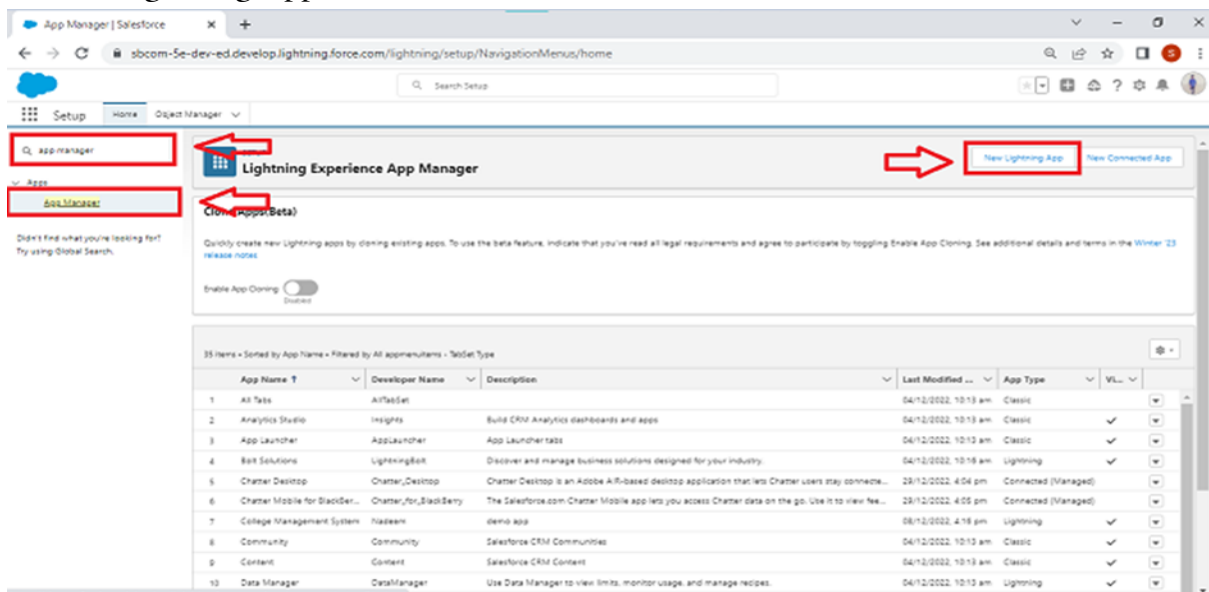


Milestone 4- The Lightning App:

Activity 1: Create a Lightning App

To create a lightning app page:

1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



- Fill the app name in app details and branding as follow
 App Name : WhatNext Vision Motors
 Developer Name : this will auto populated
 Description : Give a meaningful description

Image : optional (if you want to give any image you can otherwise not mandatory)

Primary color hex value : keep this default

2. Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.

3. To Add Navigation Items:

Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

Note: select the custom object which we have created in the previous activity.

4. To Add User Profiles:

Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

Milestone 5 : Fields & Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle Name**(Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2. Vehicle_Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)

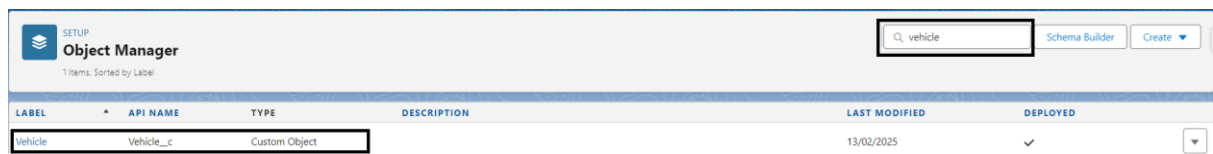
- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

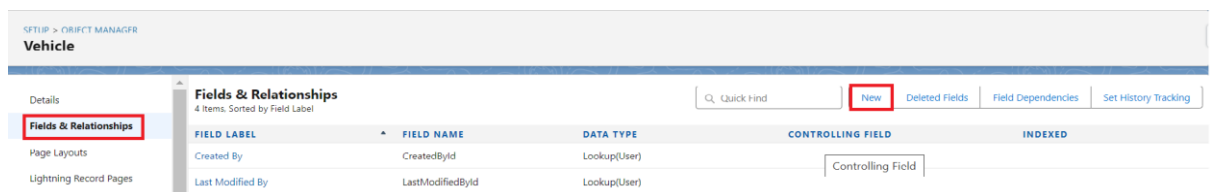
- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

Activity 1: Creating a Field in Vehicle Object

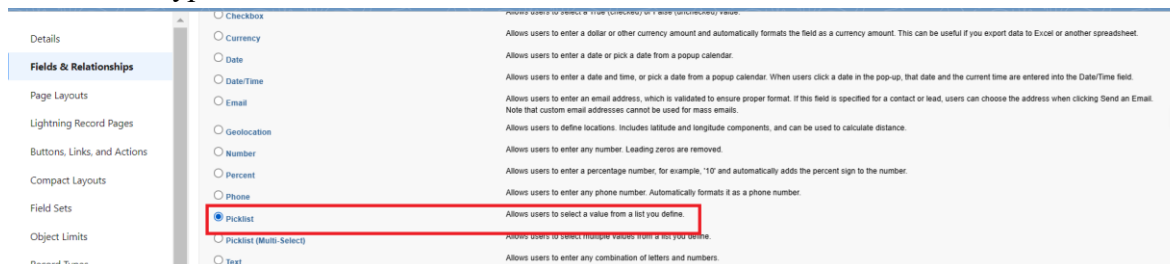
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”.



4. Click on Next

The screenshot shows the 'New Custom Field' configuration for the 'Vehicle' object. The 'Field Label' is 'Vehicle Model'. Under 'Values', the option 'Enter values, with each value separated by a new line' is selected. The text area contains 'Sedan', 'SUV', 'EV', and 'etc'. The 'Field Name' is 'Vehicle_Model'. The 'Next' button is highlighted.

5. Fill the above as following:
 - Field Label: Vehicle Modal
 - Fill PickList Values
 - Click on Next → Next → Save and new.

Activity 2 : Creating Stock Quantity field in vehicle object

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.

The screenshot shows the 'Data Type' selection screen. The 'Number' option is selected and highlighted with a red box. Other options include Checkbox, Currency, Date, Date/Time, Email, Geolocation, Percent, and Phone.

3. Click on Next

The screenshot shows the 'New Custom Field' configuration for the 'Stock Quantity' field. The 'Field Label' is 'Stock Quantity'. The 'Field Name' is 'Stock_Quantity'. The 'Next' button is highlighted.

4. Fill the above as following:
 - Field Label: Stock Quality
 - Click on Next → Next → Save and new.

Activity 3 : Creating a Lookup Relationship in Vehicle Object to Dealer Object.

1. Select DataType as Lookup Relationship.

Vehicle
New Custom Field

Step 1. Choose the field type

Specify the type of information that the custom field will contain.

Data Type

Select one of the data types below:

- ☐ None Selected
- ☐ Auto Number
- ☐ Formula
- ☐ Roll-Up Summary
- ☒ **Lookup Relationship**
- ☐ Master-Detail Relationship

Creates a relationship that links this object to another object. The relationship field allows users to click on a lookup icon to select a value from a popup list. The other object is the source of the values in the list.

Creates a special type of parent-child relationship between this object (the child, or "detail") and another object (the parent, or "master") where:

- The relationship field is required on all detail records.
- The ownership and sharing of a detail record are determined by the master record.
- When a user deletes the master record, all detail records are deleted.

Next

2. Click Next

3. Select Dealer Object Form Related Filed.

Vehicle
New Relationship

Step 2. Choose the related object

Select the other object to which this object is related.

Related To: Dealer

Previous Next Cancel

Vehicle
New Relationship

Step 3. Enter the label and name for the lookup field

Field Label: Dealer

Field Name: Dealer

Description:

Help Text:

Previous Next Cancel

Vehicle
New Relationship

Step 4. Establish field-level security for reference field

Select the profiles to which you want to grant edit access to this field via field-level security. The field will be hidden from all profiles if you do not add it to field-level security.

Field-Level Security for Profile	Visible	Read-Only
Analytics Cloud Integration User	☒	☐
Analytics Cloud Security User	☒	☐
Contract Manager	☒	☐
Cross Org Data Proxy User	☒	☐
Custom: Marketing Profile	☒	☐

Previous Next Cancel

4. Click Next and Save.

Activity 4 : Creating Price field in vehicle object

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Currency”.

The screenshot shows the Salesforce Setup interface for the 'Vehicle' object. The left sidebar contains a navigation menu with options: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, Scoping Rules, and Object Access. The main content area is titled 'Vehicle' and shows the 'Fields & Relationships' section. It lists various field types: Master Detail Relationship, External Lookup Relationship, Checkbox, Currency (selected), Date, Date/Time, Email, Geolocation, Number, Percent, Phone, Picklist, Picklist (Multi-Select), and Text. Each field type has a brief description and a 'Create' button.

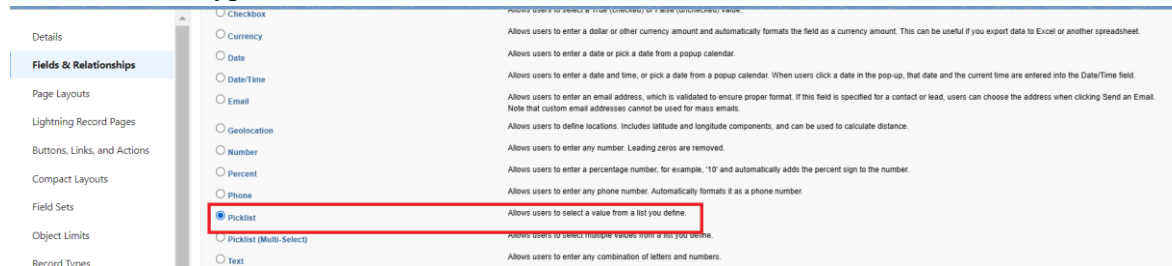
3. Click on Next

The screenshot shows the 'New Custom Field' page for the 'Vehicle' object. The page is titled 'New Custom Field' and is in 'Step 2. Enter the details'. The left sidebar is the same as the previous screenshot. The main content area contains the following fields: Field Label (Price), Length (18), Decimal Places (0), Field Name (Price), Description, and Help Text. There are also checkboxes for 'Required' (Always require a value in this field in order to save a record) and 'Auto add to custom report type' (Add this field to existing custom report types that contain this entity). The page has a 'Previous' button, a 'Next' button, and a 'Cancel' button. The bottom right corner shows the 'Einstein' logo.

4. Fill the above as following:
 - Field Label: Price
 - Click on Next → Next → Save and new.

Activity 5 : Creating Status field in vehicle object

5. Select Data type as “PickList”.



6. Click on Next
7. Fill the above as following:
 - Field Label: Status
 - Fill PickList Values: Available, Out of Stock, Discontinued
8. Click on Next → Next → Save.

Activity 6 : Creating Dealer Name field in vehicle_Dealer object

9. Repeat the steps 1 & 2 from the activity 1
10. Select Data type as “Text”.
11. Click on Next
12. Fill the above as following:
 - Field Label: Dealer_Name
13. Click on Next → Next → Save and new.

Activity 7 : Creating Dealer Location field in vehicle_Dealer object

14. Select Data type as “Text”.
15. Click on Next
16. Fill the above as following:
 - Field Label: Dealer_location
17. Click on Next → Next → Save and new.

Activity 8 : Creating Dealer Code field in vehicle_Dealer object

18. Select Data type as “Auto Number”.
19. Click on Next
20. Fill the above as following:
 - Field Label: Dealer_Code
21. Click on Next → Next → Save and new.

Activity 9 : Creating Phone field in vehicle_Dealer object

22. Select Data type as “Phone”.
23. Click on Next
24. Fill the above as following:
 - Field Label: Phone
25. Click on Next → Next → Save and new.

Activity 10 : Creating Email field in vehicle_Dealer object

26. Select Data type as “Email”.
27. Click on Next
28. Fill the above as following:
 - Field Label: Email
29. Click on Next → Next → Save.

Activity 11 : Creating Vehicle Customer field in vehicle_Order object

30. Repeat the steps 1 & 2 from the activity 1
31. Select Data type as “Lookup Relationship”.
32. Click on Next
33. Fill the above as following:
 - Field Label: Vehicle_Customer
34. Click on Next → Next → Save & New.

Activity 12 : Creating Vehicle field in vehicle_Order object

35. Select Data type as “Lookup Relationship”.
36. Click on Next
37. Fill the above as following:
 - Field Label: Vehicle
38. Click on Next → Next → Save & New.

Activity 13 : Creating Order Date field in vehicle_Order object

39. Select Data type as “Date”.
40. Click on Next
41. Fill the above as following:
 - Field Label: Order_date
42. Click on Next → Next → Save & New.

Activity 14 : Creating Status field in vehicle_Order object

43. Select Data type as “PickList”.
44. Click on Next
45. Fill the above as following:
 - Field Label: Status
 - Fill PickList Values: Pending, Confirmed, Delivered, Canceled
46. Click on Next → Next → Save.

Activity 15 : Creating Customer Name field in Vehicle_Customer object

47. Repeat the steps 1 & 2 from the activity 1
48. Select Data type as “Text”.
49. Click on Next
50. Fill the above as following:
 - Field Label: Customer_Name
51. Click on Next → Next → Save & New.

Activity 16 : Creating Email field in Vehicle_Customer object

52. Select Data type as “Email”.
53. Click on Next
54. Fill the above as following:
 - Field Label: Email
55. Click on Next → Next → Save & New

Activity 17 : Creating Phone field in Vehicle_Customer object

56. Select Data type as “Phone”.
57. Click on Next
58. Fill the above as following:
 - Field Label: Phone
59. Click on Next → Next → Save & New

Activity 18 : Creating Address field in Vehicle_Customer object

60. Select Data type as “Text”.
61. Click on Next
62. Fill the above as following:
 - Field Label: Address
63. Click on Next → Next → Save & New

Activity 19 : Creating Preferred Vehicle Type field in Vehicle_Customer object

64. Select Data type as “Picklist”.
65. Click on Next
66. Fill the above as following:
 - Field Label: Preferred_Vehicle_Type
 - Fill PickList Values: Sedan, SUV, EV, etc.
67. Click on Next → Next → Save.

Activity 20 : Creating Vehicle Customer field in Vehicle_Test_drive object

68. Repeat the steps 1 & 2 from the activity 1
69. Select Data type as “Lookup Relationship”.
70. Click on Next
71. Fill the above as following:
 - Field Label: Vehicle _Customer
72. Click on Next → Next → Save & New.

Activity 21 : Creating Vehicle field in Vehicle_Test_drive object

73. Select Data type as “Lookup Relationship”.
74. Click on Next
75. Fill the above as following:
 - Field Label: Vehicle
76. Click on Next → Next → Save & New.

Activity 22 : Creating Test_Drive_Date field in Vehicle_Test_drive object

77. Select Data type as “Date”.
78. Click on Next
79. Fill the above as following:
 - Field Label: Test_Drive_Date
80. Click on Next → Next → Save & New.

Activity 23 : Creating Status field in Vehicle_Test_drive object

81. Select Data type as “Picklist”.
82. Click on Next
83. Fill the above as following:
 - Field Label: Status
 - Fill the Picklist value: Scheduled, Completed, Canceled
84. Click on Next → Next → Save.

Activity 24 : Creating Vehicle Customer field in Vehicle_Service_Request object

85. Repeat the steps 1 & 2 from the activity 1
86. Select Data type as “Lookup Relationship”.
87. Click on Next
88. Fill the above as following:
 - Field Label: Vehicle_Customer
89. Click on Next → Next → Save & New.

Activity 25 : Creating Vehicle field in Vehicle_Service_Request object

90. Select Data type as “Lookup Relationship”.
91. Click on Next
92. Fill the above as following:
 - Field Label: Vehicle
93. Click on Next → Next → Save & New.

Activity 26 : Creating Service Date field in Vehicle_Service_Request object

94. Select Data type as “Date”.
95. Click on Next
96. Fill the above as following:
 - Field Label: Service_Date
97. Click on Next → Next → Save & New.

Activity 27 : Creating Issue Description field in Vehicle_Service_Request object

98. Select Data type as “Text”.
99. Click on Next
100. Fill the above as following:
 - Field Label: Issue_Description
101. Click on Next → Next → Save & New.

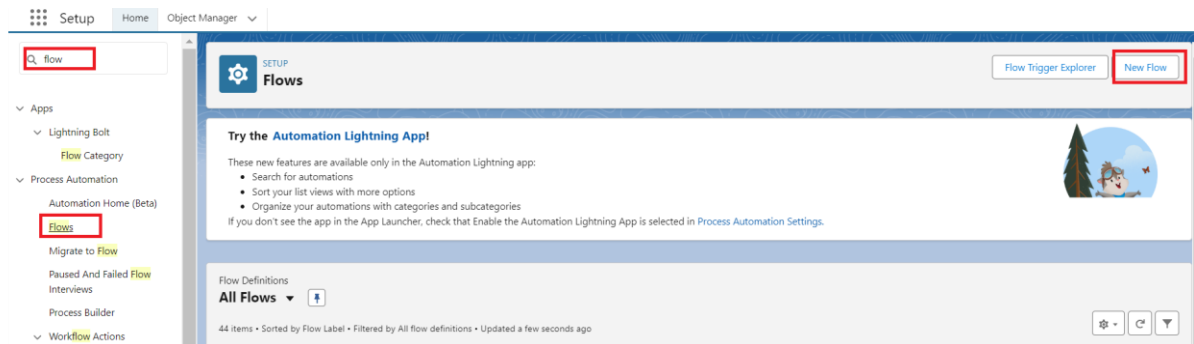
Activity 28 : Creating Status field in Vehicle_Service_Request object

102. Select Data type as “Picklist”.
103. Click on Next
104. Fill the above as following:
 - Field Label: Status
 - Fill Picklist value: Requested, In Progress, Completed
105. Click on Next → Next → Save & New.

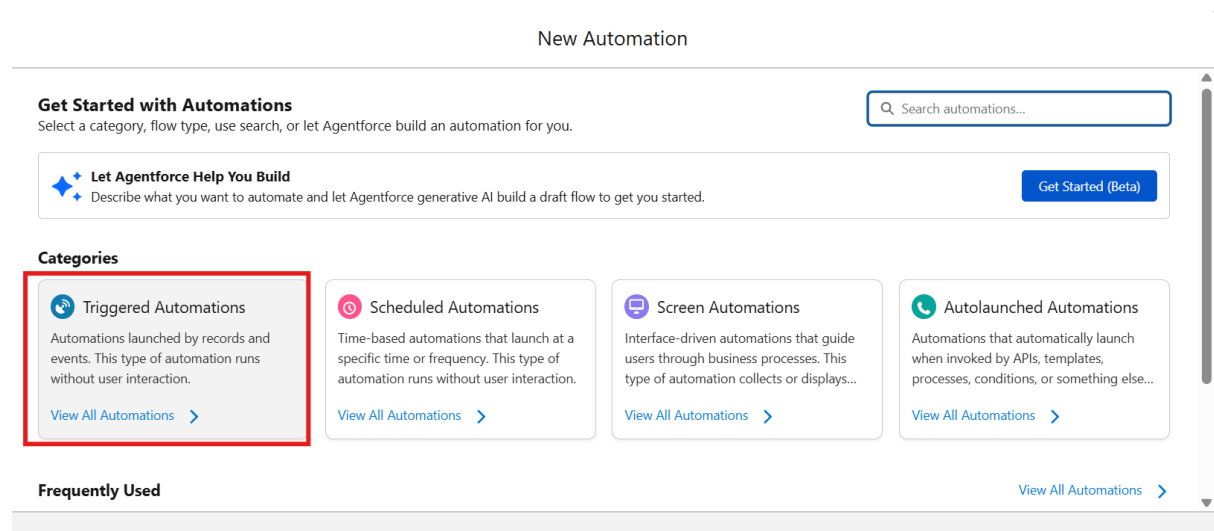
MileStone 6 :

Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer's location

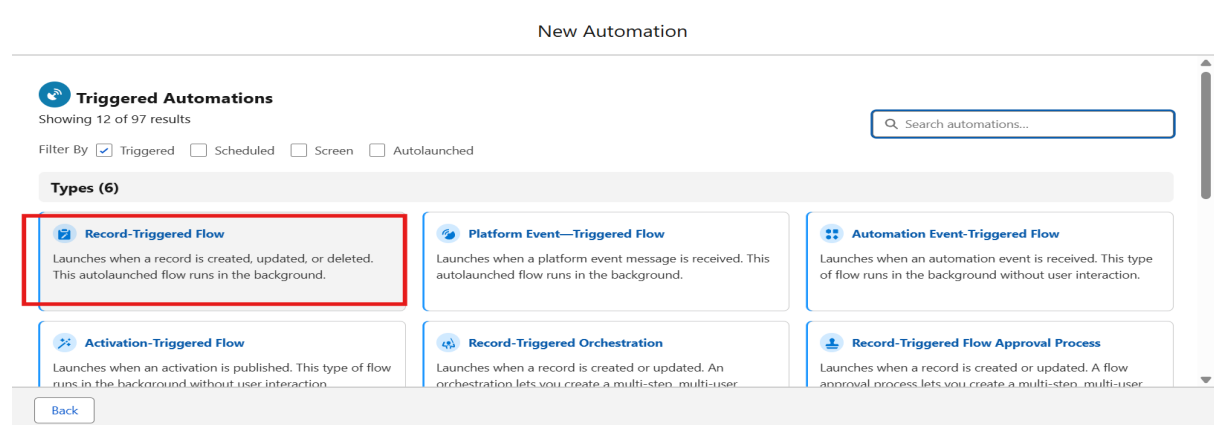
Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Triggered Automation From Flow.



Step 3 : Select Record-Triggered Flow



Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :

All Conditions Are Met (AND)

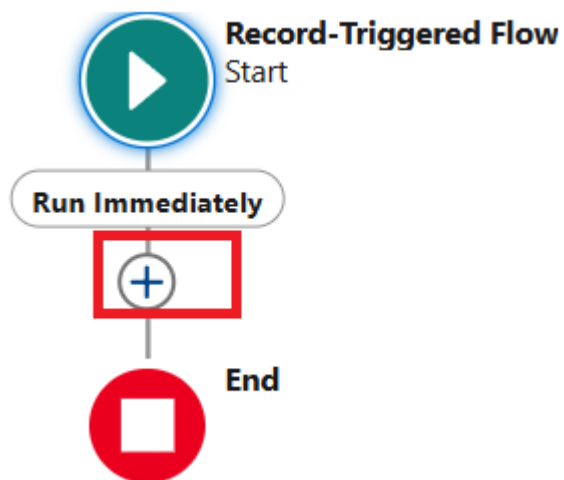
Filed : Status__c

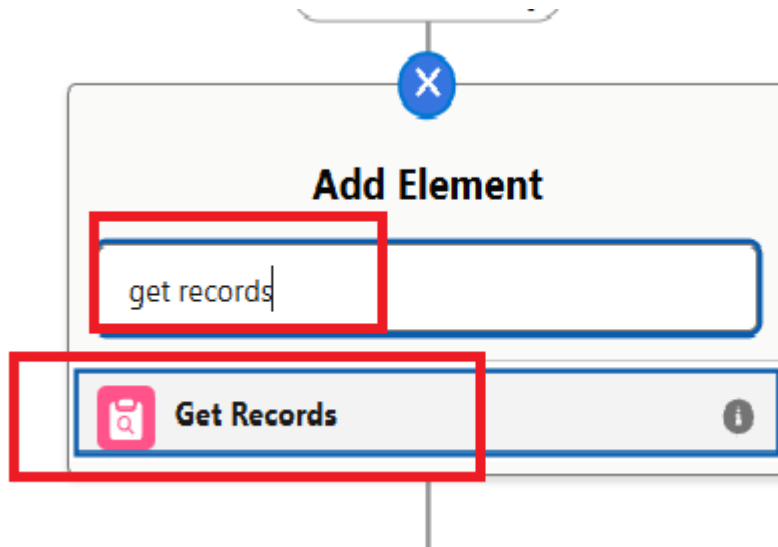
Operator : Equals

Value : Pending

The screenshot displays the Salesforce Flow Builder interface. On the left, a flow diagram shows a 'Start' node (Record-Triggered Flow) leading to a 'Run Immediately' node, followed by two 'Get Records' nodes: 'Get Customer Information' and 'Get Nearest Dealer'. On the right, the 'Configure Start' panel is open. The 'Object' is set to 'Vehicle Order'. Under 'Configure Trigger', the option 'A record is created' is selected. The 'Set Entry Conditions' section shows a condition requirement of 'All Conditions Are Met (AND)'. A single condition is defined with the field 'Vehicle_Customer__c', the operator 'Is Null', and the value 'false'.

Step 5 : Click + → Select **Get Records**





Step 6 : **Label** : Get Customer Information

Object : Vehicle Customer

Condition :

Field : Id

Operator : Equals

Value : {!\$Record.Vehicle_Customer__c}

 Get Records

*Label *API Name 

Get Customer Information. Get_Customer_Information

Description

Get Records of This Object

*Object

Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements

All Conditions Are Met (AND) 

Field	Operator	Value
Id	Equals 	Aa \$Record > Vehicle Customer 

[+ Add Condition](#)

Sort Vehicle Customer Records

Step 7 : Click + → Select **Get Records**

Label : Get Nearest Dealer

Object : Vehicle Dealer

Condition :

Field : Dealer_Location__c

Operator : Equals

Value : {!Get_Customer_Information.Address__c}



Get Records

* Label

Get Nearest Dealer

* API Name ⓘ

Get_Nearest_Dealer

Description

Get Records of This Object

* Object

Vehicle Dealer

Filter Vehicle Dealer Records

Condition Requirements

All Conditions Are Met (AND) ▼

Field

Dealer_Location__c

Operator

Equals ▼

Value

 Vehicle Customer from Get_Cu... ✕

+ Add Condition

Step 8 : Click + → Select **Update Records**

Label : Assign Dealer to Order

*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection

Select Record(s) to Update : {!Get_Nearest_Dealer}

 Update Records >

* Label

Assign Dealer to Order

* API Name ⓘ

Assign_Dealer_to_Order

Description

* How to Find Records to Update and Set Their Values

☐ Use the vehicle order record that triggered the flow

☐ Update records related to the vehicle order record that triggered the flow

☒ Use the IDs and all field values from a record or record collection

☐ Specify conditions to identify records, and set fields individually

Select Record(s) to Update

Record or Record Collection

 Vehicle Dealer from Get Nearest Dealer X

Make sure that each record has an ID. Otherwise the flow can't find the records to update, and it fails. ⓘ

Step 9 : Click Save and Give label Name and Activate Flow.

Label Name : Auto Assign Dealer

Save as

Save As

A New Version

* Flow Label

Auto Assign Dealer

* Flow API Name ⓘ

Auto_Assign_Dealer

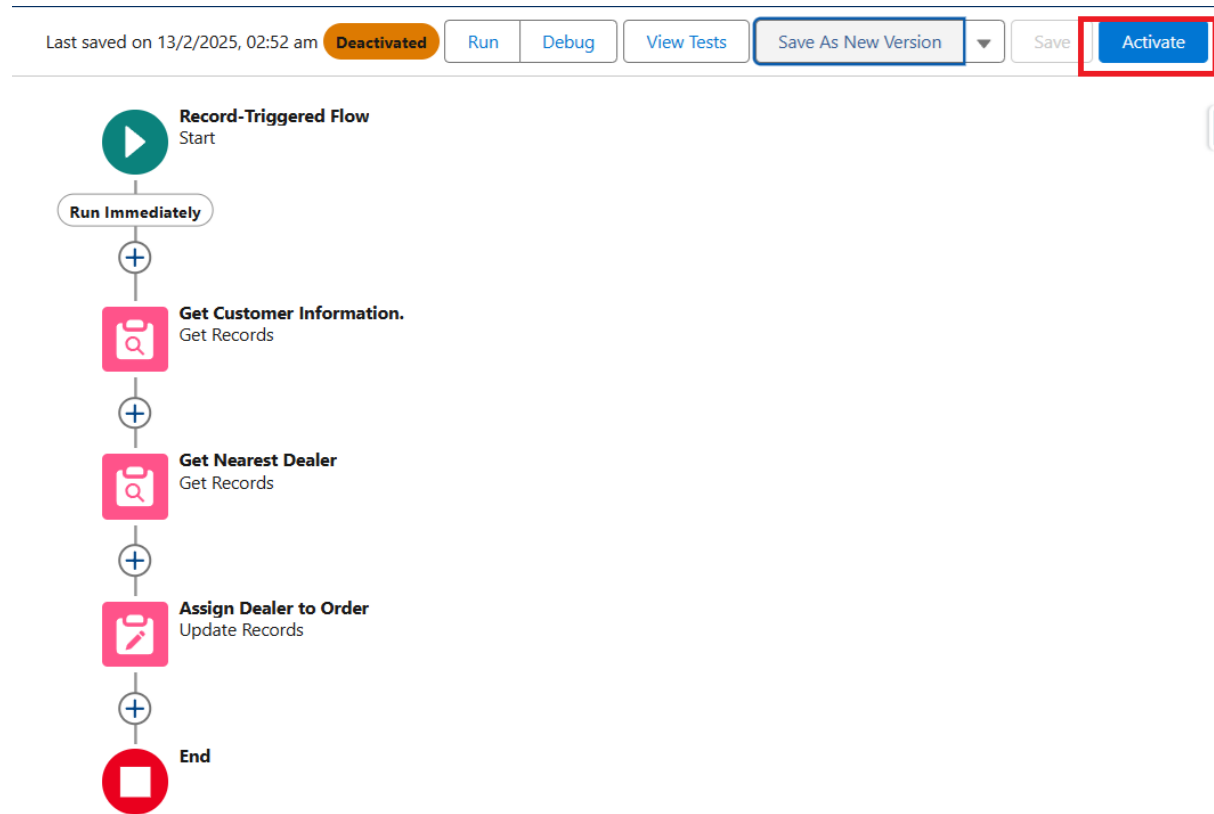
Description

Show Advanced

Cancel

Save

Step 10 : Activate Flow



Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow

New Automation

Triggered Automations
Showing 12 of 97 results

Filter By ☒ Triggered ☐ Scheduled ☐ Screen ☐ Autolaunched

Types (6)

Record-Triggered Flow
Launches when a record is created, updated, or deleted. This autolaunched flow runs in the background.

Platform Event—Triggered Flow
Launches when a platform event message is received. This autolaunched flow runs in the background.

Automation Event-Triggered Flow
Launches when an automation event is received. This type of flow runs in the background without user interaction.

Activation-Triggered Flow
Launches when an activation is published. This type of flow runs in the background without user interaction.

Record-Triggered Orchestration
Launches when a record is created or updated. An orchestration lets you create a multi-step, multi-user

Record-Triggered Flow Approval Process
Launches when a record is created or updated. A flow approval process lets you create a multi-step, multi-user

[Back](#)

Step 2 : Select Vehicle Test Drive Object

Trigger the Flow When : A record is created or updated

Set Entry Condition :

All Conditions Are Met (AND)

Filed : Status__c

Operator : Equals

Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

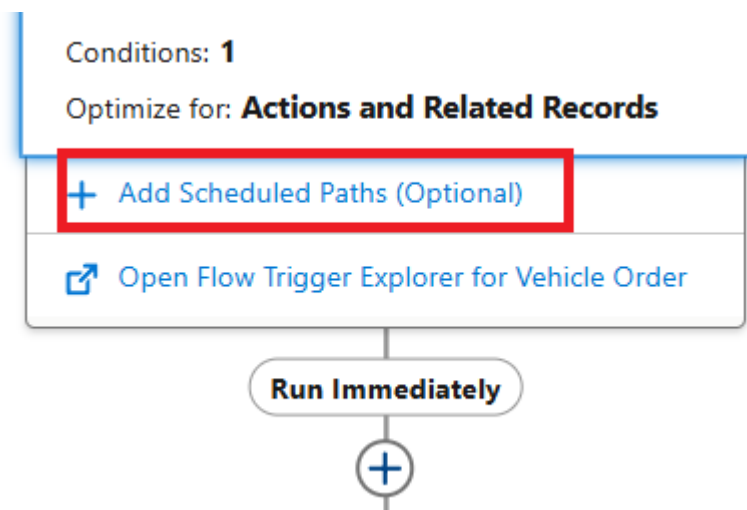
Label: Reminder Before Test Drive.

Time Source: Test_Drive_Date__c

Offset Number: 1.

Offset Options: Days Before.

Click Done.



Configure Scheduled Paths

SCHEDULED PATHS
+

Run Immediately

Reminder Before Test Drive.

Add a scheduled path if you want a flow path to run a certain amount of time after the triggering record is created or updated. That time can be based on the triggering event or on a specified date or date/time field in the record. A separate flow interview runs for each scheduled path. Each scheduled path interview is queued to run at the scheduled time, but can be delayed until system resources become available. Path labels appear on connectors in the canvas.

SCHEDULED PATH DETAILS
Delete Path

* Path Label
Reminder Before Test Drive.

* API Name
Reminder_Before_Test_Drive

* Time Source
Vehicle_Test_Drive__c: Test Drive Date

* Offset Number
1

* Offset Options
Days Before

> Show advanced options

Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!\$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

Get Records

* Label
Get Customer Information

* API Name
Get_Customer_Information

Description

Get Records of This Object

Object
Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements
All Conditions Are Met (AND)

Field
Id

Operator
Equals

Value
\$Record > Vehicle Customer

+ Add Condition

Sort Vehicle Customer Records

Step 5: Send Reminder Email

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

 **Send Email**

* Label

Send Test Drive Reminder.

* API Name ⓘ

Send_Test_Drive_Reminder

Description

 **Send Email** ⓘ
emailSimple-emailSimple

7. Body : Create Variable

New Resource

* Resource Type

Select...

Variable
Store a value that can be used and changed throughout the flow.

Constant
Store a value that can be used but not changed throughout the flow.

Formula
Calculate a value when the formula is used in the flow.

Text Template
Store text that can be used and changed throughout the flow.

Stage
Identify different phases in the flow to track user progress.

8 . Api Name : EmailSent

Edit Text Template

* API Name ⓘ

EmailSent

Description

* Body ⓘ

Insert a resource...

View as Rich Text ▾

Dear {!Get_Customer_Information.Name},

This is a reminder that your test drive for {!\$Record.Vehicle__c} is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email]. Thank you, WhatNext Vision Motors Team"

Salesforce Sans ▾

12 ▾

■ ▾

B

I

U

☰

☷

+☰

+☷

☰

☷

☰

☷

🔗

🖼️

🔗

Cancel

Done

⚡ Send Email

Aa Body ⓘ	📧 EmailSent X	✓ Not Included
Aa CC Recipient Address List		Not Included
Aa Email Template ID		Not Included
🔗 Log Email on Send		Not Included
Aa Recipient Address Collection		Not Included

Step 6 : Click save

Label Name : Test Drive Reminder

Save as

Save As

A New Version

Flow Label

Test Drive Reminder

* Flow API Name

Test_Drive_Reminder

Description

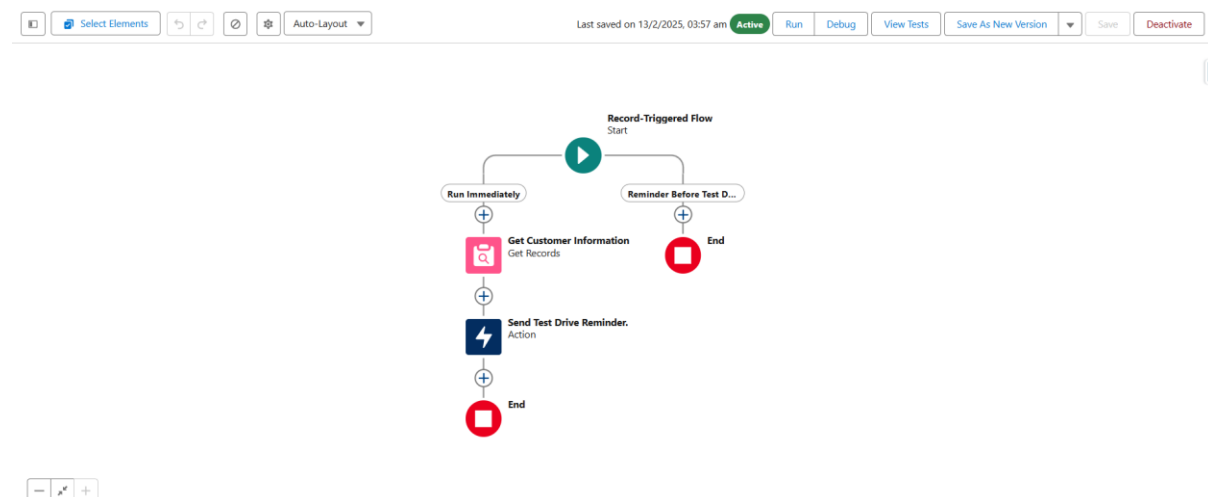
Show Advanced

Cancel Save

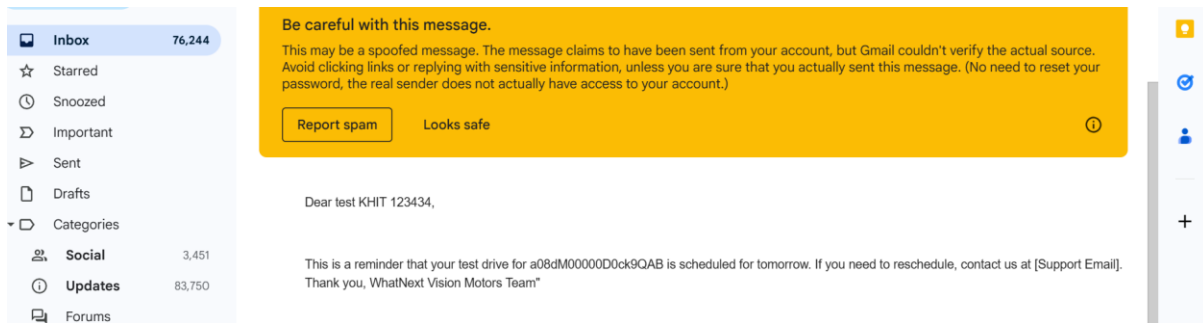
Step 7 : Activate Flow

Run Debug View Tests Save As New Version Save Activate

Flow Page After Creation :



Test your Flow :



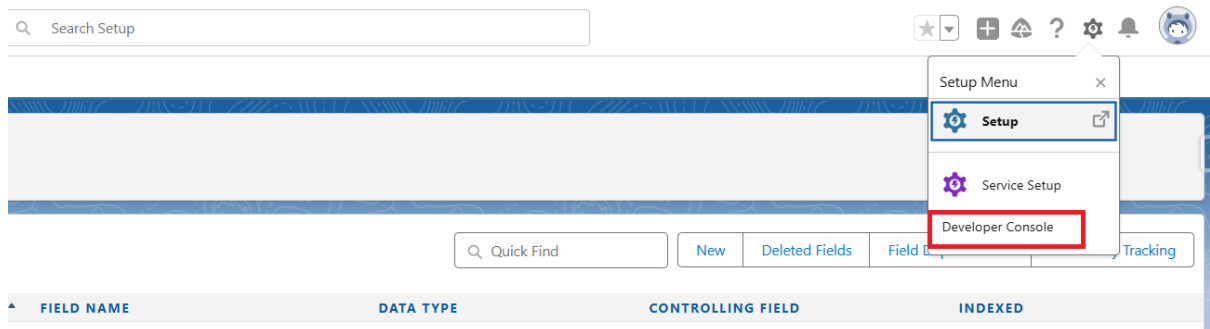
Send Email

A_a Recipient Address List A_a ...er from Get Customer Information > Email ×	✓ Included
A_a Recipient ID Vehicle Customer from Get Customer Information > Email	Not Included
A_a Related Record ID	Not Included
🔗 Rich-Text-Formatted Body True ×	✓ Included
A_a Sender Email Address	Not Included
A_a Sender Type	Not Included
A_a Subject Reminder: Your Test Drive is Tomorrow! 🔍	✓ Included
🔗 Use Line Breaks True ×	✓ Included

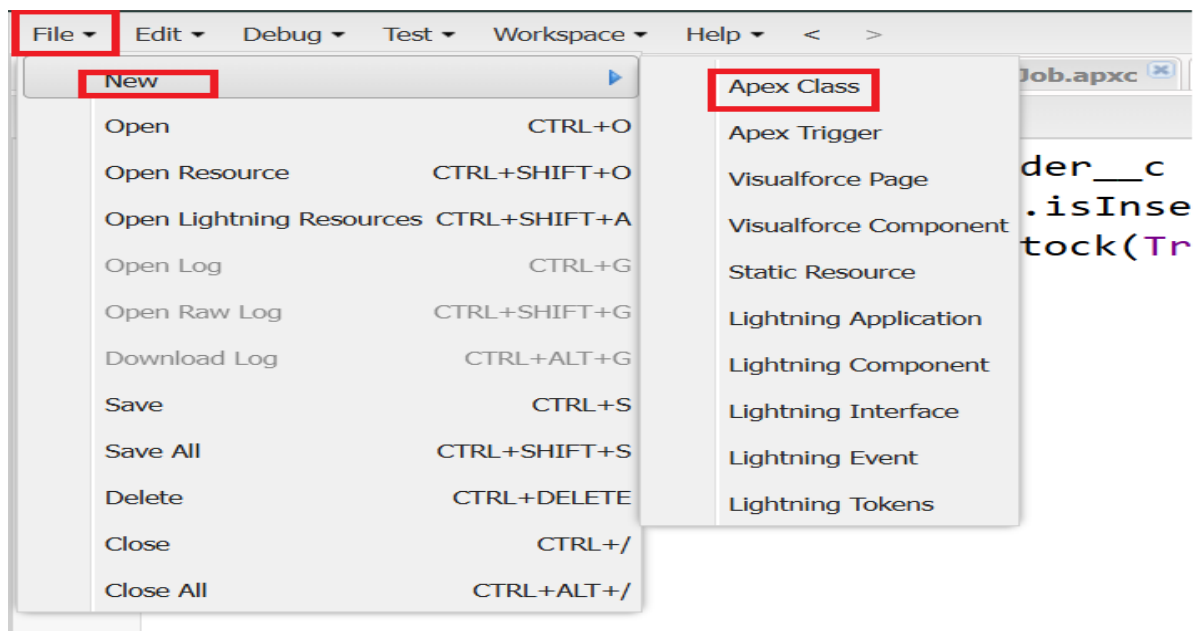
MileStone 7 :

Create Apex and Trigger Batch Jobs :

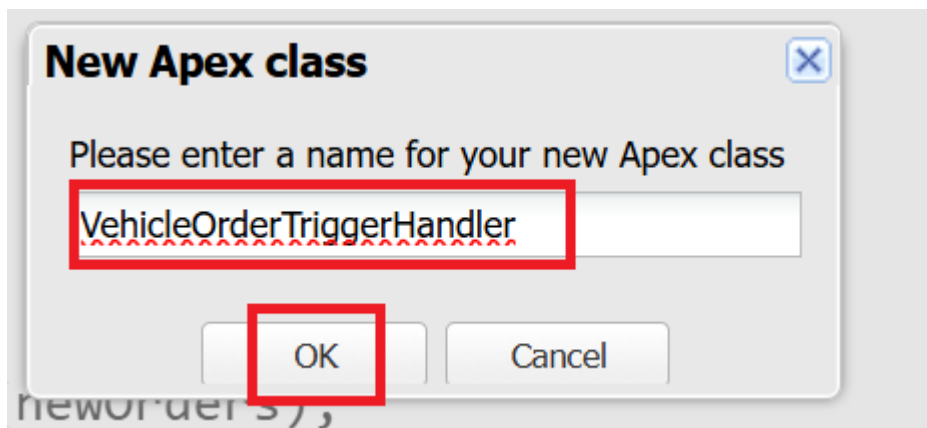
Step 1 : Click Developer Console From Gear icon



Step 2 : click File then Select New then Select Apex.



Step 3 : Give Name For Apex Class



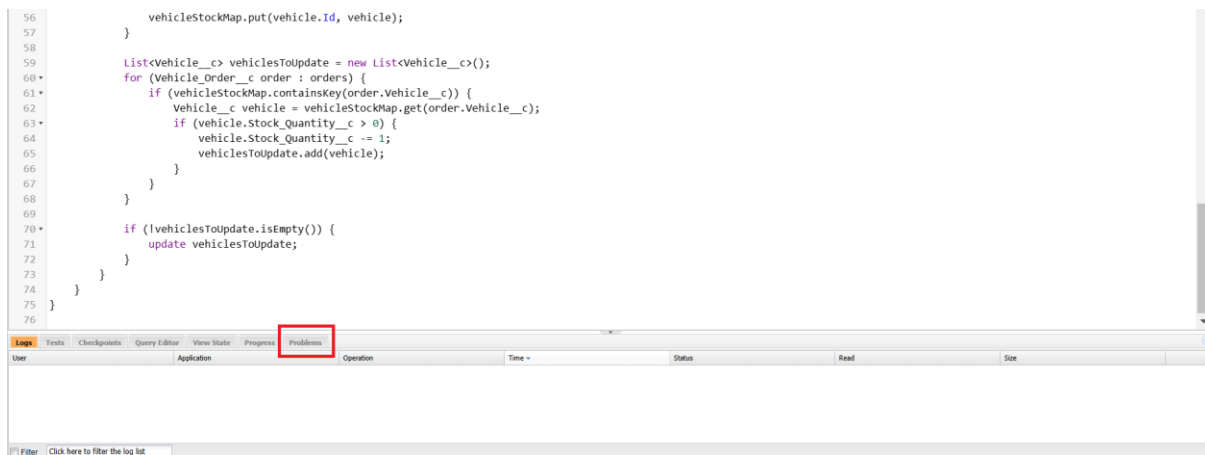
Step 4 : Write Apex Code

```
1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
4         if (isBefore) {
5             if (isInsert || isUpdate) {
6                 preventOrderIfOutOfStock(newOrders);
7             }
8         }
9
10        if (isAfter) {
11            if (isInsert || isUpdate) {
12                updateStockOnOrderPlacement(newOrders);
13            }
14        }
15    }
16
17    // Method to prevent orders when the vehicle is out of stock
18    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
19        Set<Id> vehicleIds = new Set<Id>();
20        for (Vehicle_Order__c order : orders) {
21            if (order.Vehicle__c != null) {
22                vehicleIds.add(order.Vehicle__c);
23            }
24        }
25
26        if (!vehicleIds.isEmpty()) {
27            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
28            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
29                vehicleStockMap.put(vehicle.Id, vehicle);
30            }
31
32            for (Vehicle_Order__c order : orders) {
33                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
34                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
35                    if (vehicle.Stock_Quantity__c <= 0) {
36                        order.addError('This vehicle is out of stock. Order cannot be placed.');
```

```

56         vehicleStockMap.put(vehicle.Id, vehicle);
57     }
58
59     List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
60     for (Vehicle_Order__c order : orders) {
61         if (vehicleStockMap.containsKey(order.Vehicle__c)) {
62             Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
63             if (vehicle.Stock_Quantity__c > 0) {
64                 vehicle.Stock_Quantity__c -= 1;
65                 vehiclesToUpdate.add(vehicle);
66             }
67         }
68     }
69
70     if (!vehiclesToUpdate.isEmpty()) {
71         update vehiclesToUpdate;
72     }
73 }
74
75 }
76

```



The screenshot shows an IDE window with a Java file. The code is a class with a method that updates vehicle stock. The 'Problems' tab in the bottom panel is highlighted with a red box. The IDE interface includes tabs for Logs, Tests, Checkpoints, Query Editor, View State, Progress, and Problems. Below the Problems tab is a table with columns: User, Application, Operation, Time, Status, Read, and Size. At the bottom, there is a 'Filter' button and a link 'Click here to filter the log list'.

Source Code :

```
public class VehicleOrderTriggerHandler {
```

```
    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean
isUpdate) {
```

```
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }
    }
```

```
        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }
}
```

// Method to prevent orders when the vehicle is out of stock

```
private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
```

```
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null) {
            vehicleIds.add(order.Vehicle__c);
        }
    }
}
```

```
    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
    }
```

```

        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0) {
                    order.addError("This vehicle is out of stock. Order cannot be placed.");
                }
            }
        }
    }
}

```

```

// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();

```

```

        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }
    }
}

```

```

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

```

```

            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        vehicle.Stock_Quantity__c -= 1;
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }
        }
    }
}

```

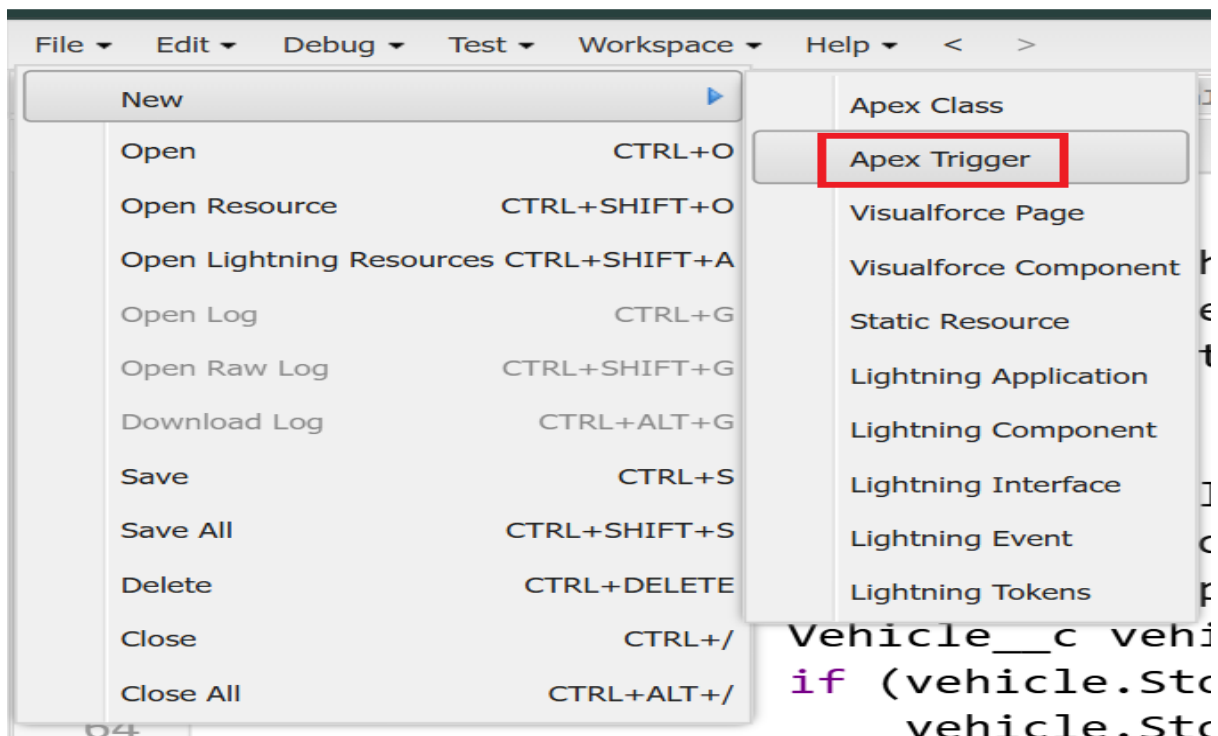
```

    if (!vehiclesToUpdate.isEmpty()) {
        update vehiclesToUpdate;
    }
}
}
}
}

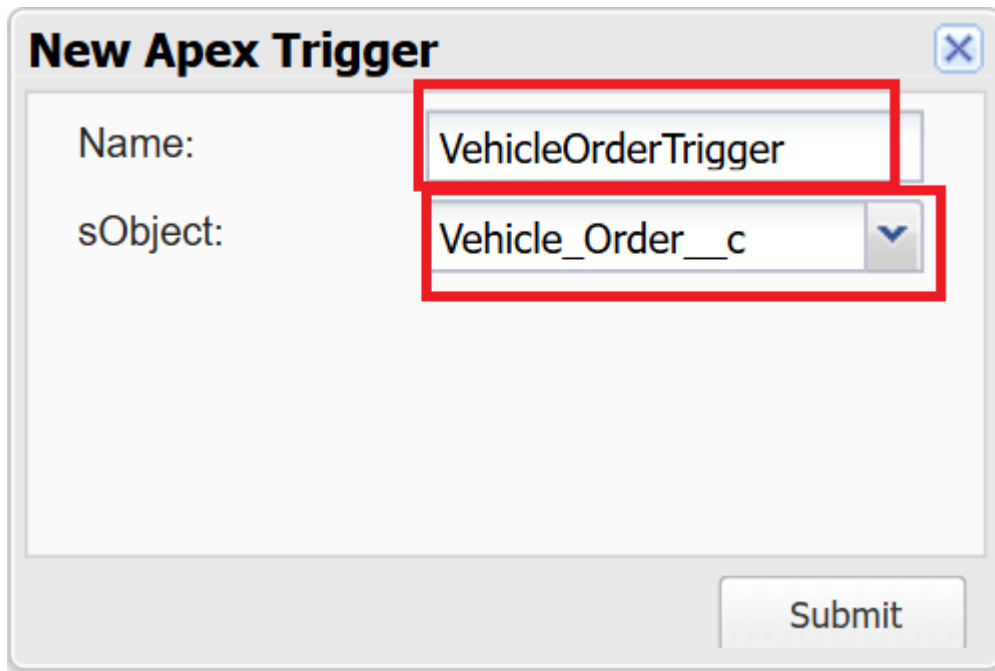
```

The screenshot shows a Salesforce 'Vehicle Order' form with the title 'VC-0003'. The form includes fields for 'Vehicle Customer' (test KHIT 123434), 'Vehicle' (1erwe), 'Order Date' (21/02/2025), and 'Status' (Pending). The owner is 'Abdul Trainer'. A red error message box is displayed in the center, stating 'We hit a snag.' and 'Review the errors on this page.' with a bullet point: 'This vehicle is out of stock. Order cannot be placed.' Below the error message are 'Cancel' and 'Save' buttons.

Step 5 : Write Trigger Handler

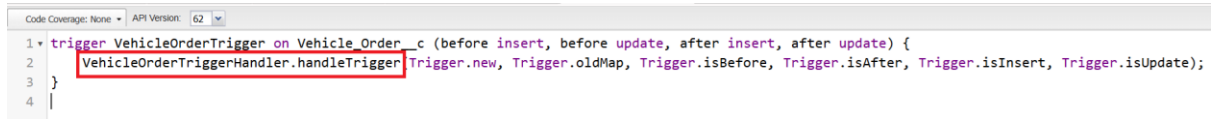


Step 6 : Writer Trigger Class Name and Select Vehicle Order Object



The 'New Apex Trigger' dialog box is shown. The 'Name' field contains 'VehicleOrderTrigger' and the 'sObject' dropdown menu is set to 'Vehicle_Order__c'. Both fields are highlighted with red rectangles. A 'Submit' button is located at the bottom right.

Step 7 : Call Apex Class in Trigger Class

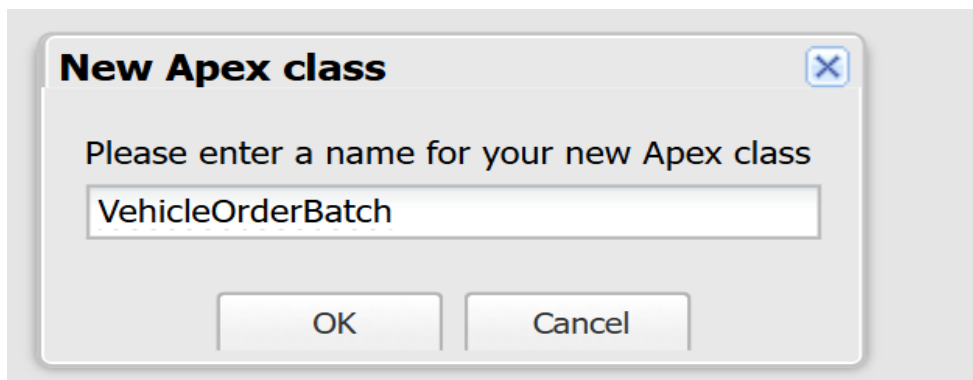


```
1 trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {  
2     VehicleOrderTriggerHandler.handleTrigger(trigger.new, trigger.oldMap, trigger.isBefore, trigger.isAfter, trigger.isInsert, trigger.isUpdate);  
3 }  
4
```

Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.



The 'New Apex class' dialog box is shown. It prompts the user to 'Please enter a name for your new Apex class'. The text 'VehicleOrderBatch' is entered in the input field. 'OK' and 'Cancel' buttons are at the bottom.

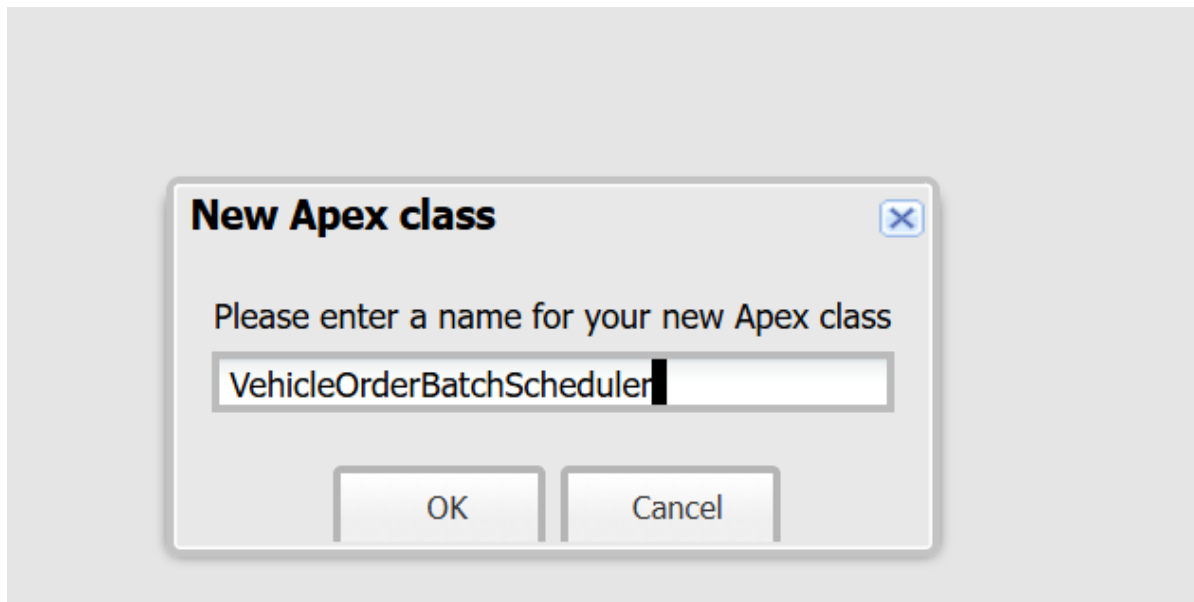
Code Coverage: None ▾ API Version: 62 ▾

1 ▾ global class VehicleOrderBatch implements Database.Batchable<SObject> {
2
3 ▾ global Database.QueryLocator start(Database.BatchableContext bc) {
4 ▾ return Database.getQueryLocator([
5 SELECT Id, Status__c, Vehicle__c
6 FROM Vehicle_Order__c
7 WHERE Status__c = 'Pending'
8]);
9 }
10
11 ▾ global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
12 Set<Id> vehicleIds = new Set<Id>();
13
14 ▾ for (Vehicle_Order__c order : orderList) {
15 ▾ if (order.Vehicle__c != null) {
16 vehicleIds.add(order.Vehicle__c);
17 }
18 }
19
20 ▾ if (!vehicleIds.isEmpty()) {
21 Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
22 ▾ for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
23 vehicleStockMap.put(vehicle.Id, vehicle);
24 }
25
26 List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
27 List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
28
29 ▾ for (Vehicle_Order__c order : orderList) {
30 ▾ if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31
32 ▾ if (vehicleStockMap.containsKey(order.Vehicle__c)) {
33 Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
34 if (vehicle.Stock_Quantity__c > 0) {
35 order.Status__c = 'Confirmed';
36 vehicle.Stock_Quantity__c -= 1;
37 ordersToUpdate.add(order);
38 vehiclesToUpdate.add(vehicle);
39 }
40 }
41 ▾ if (!ordersToUpdate.isEmpty()) {
42 update ordersToUpdate;
43 }
44
45 ▾ if (!vehiclesToUpdate.isEmpty()) {
46 update vehiclesToUpdate;
47 }
48 }
49 }
50
51 ▾ global void finish(Database.BatchableContext bc) {
52 System.debug('Vehicle order batch job completed.');53 }
54 }
}

Logs Tests Checkpoints Query Editor View State Progress Problems

UserApplicationOperation

Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class

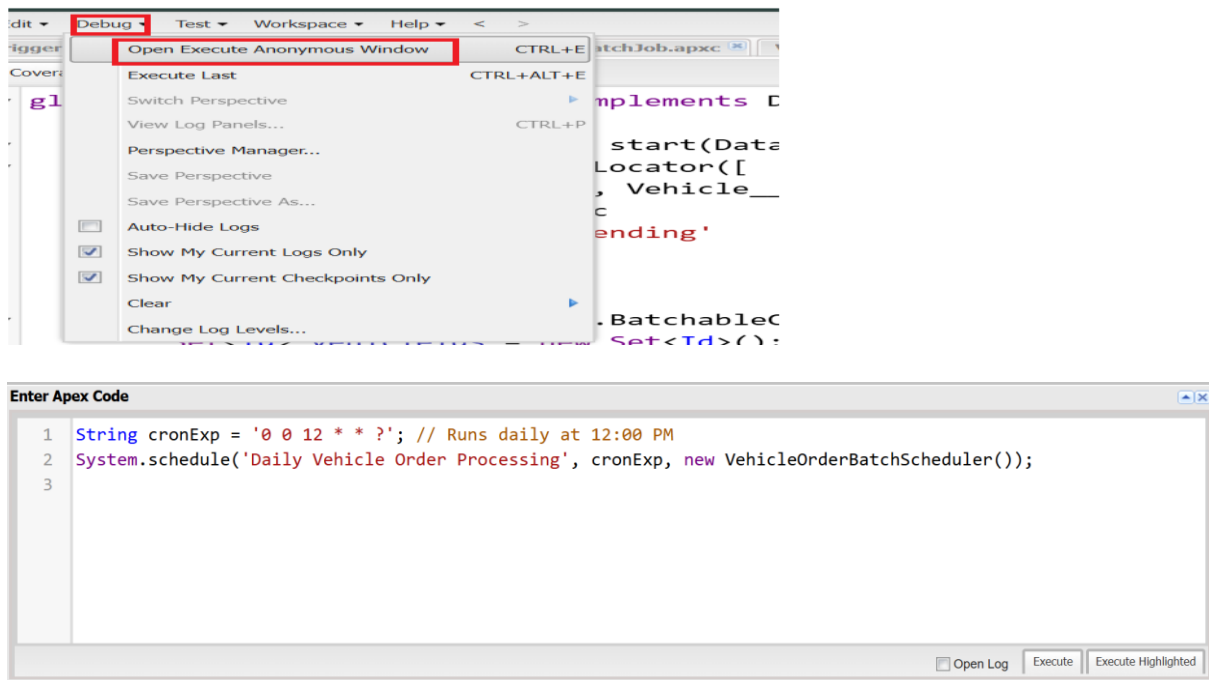


```
OrderTriggerHandler.apxc | OrderTrigger.apxt | InventoryBatchJob.apxc | VehicleOrderTriggerHandler.apxc | VehicleOrderTrigger.apxt | VehicleOrderBatchJob.apxc
Code Coverage: None | API Version: 62
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50);
5     }
6 }
7
```

Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Step 10 : Schedule the Batch Job To run the batch job every night at midnight :

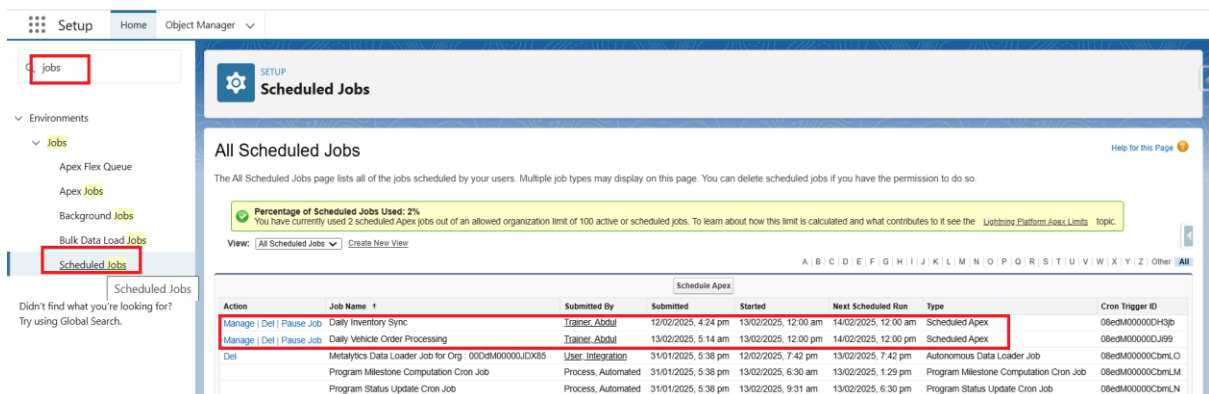


Source Code :

String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM

System.schedule('Daily Vehicle Order Processing', cronExp, new
VehicleOrderBatchScheduler());

Step 11 : You Can Check where this Schedule Job is Running



Thank You